1. Real-Time Sign Language Word Gesture Detection and On-Screen Captioning Using Computer Vision

2. Problem Statement:

Deaf and hard-of-hearing individuals face communication barriers during everyday interactions—such as at school offices, retail counters, or clinics—when conversing with people who do not know sign language. Currently, individuals rely on typing notes on a smartphone or using slow letter-by-letter fingerspelling charts, which disrupts natural face-to-face communication. Computer vision addresses this by using a standard laptop or PC webcam to detect whole-word hand gestures in real time and automatically overlay the translated word as an on-screen caption. By recognizing 8 high-frequency static word gestures (*Hello / Palm, Stop, Yes / Like, No / Dislike, Peace / Thank You, OK, Help / Call, Mute / Quiet*) directly from live video, the system provides instant visual subtitles without requiring specialized wearable sensors.

3. Objectives and Expected Outcomes:

Classification Performance: Classify 8 static word gestures with a macro-F1 score of at least 0.85 on a held-out test set spanning plain and cluttered indoor backgrounds.

Real-Time Inference Speed: Process live webcam frames and render on-screen word captions at 15 FPS or higher on a standard laptop CPU.

Method Comparison: Quantitatively evaluate the accuracy and latency trade-offs between a classical computer vision baseline and our deep learning / landmark main pipeline.

Minimum Viable Version (Fallback Outcome): If the main method fails to reach target accuracy or speed in time, we will deliver a working YCrCb skin-mask + contour convexity + SVM baseline that captions 5 static word gestures live on a webcam against a plain background.

4. Proposed Approach and Algorithms

Our system operates across five sequential stages (Acquisition → Preprocessing → Features/Model → Post-Processing → Output):

1. **Acquisition:** Capture 640×480 BGR video frames from a standard webcam at 30 FPS (or read static RGB test images from disk).
2. **Preprocessing:** Resize frames to target input dimensions, apply a 5×5 Gaussian blur to suppress camera sensor noise, and convert colour spaces (BGR -> YCrCb for the baseline; BGR -> RGB with $[0,1]$ normalization for the main method).
3. **Features or Model:**

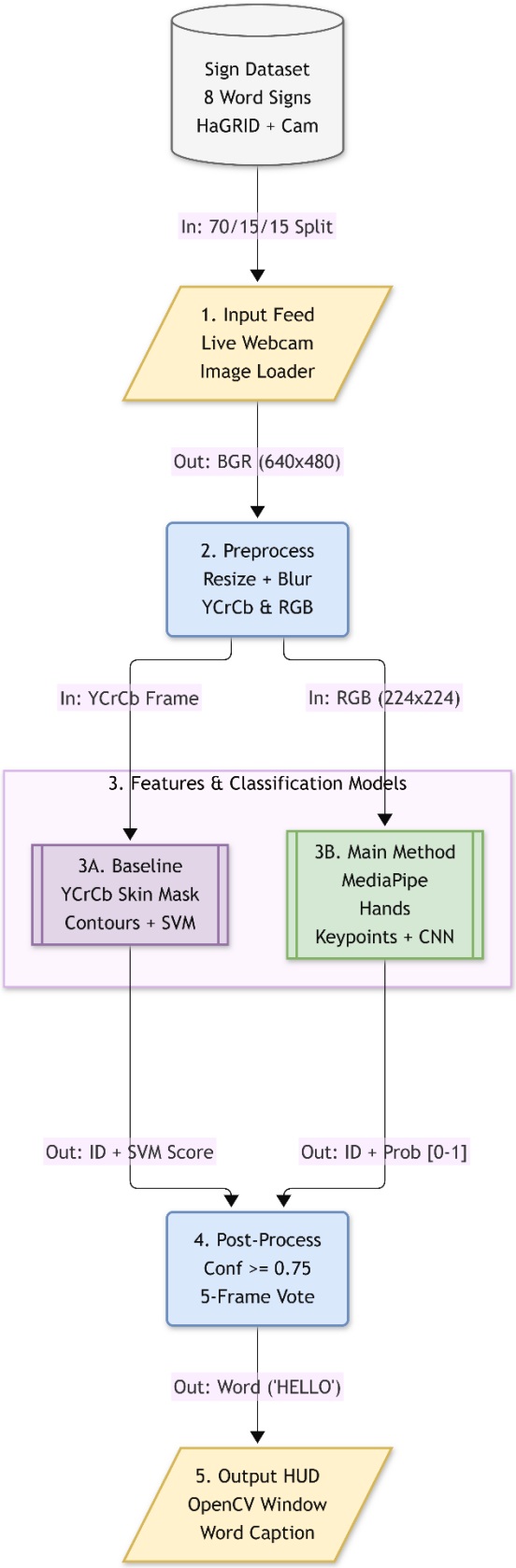
- **Baseline Method (Skin Mask + Contour/Convexity Features + SVM):** Threshold the YCrCb image to segment skin pixels, apply morphological closing, extract the largest hand contour, and compute **7 Hu Moments + Convexity Defect counts** to classify gestures with a **Support Vector Machine (SVM)**. *Why:* It requires no GPU training, can be implemented in a few days, and establishes a geometric baseline.
- **Main Method (MediaPipe Hand Landmarks / Fine-Tuned YOLOv11n Classifier):** Extract 21 normalized 3D hand-knuckle coordinates (x, y, z) via **MediaPipe Hands** fed into a lightweight MLP classifier (or fine-tune **YOLOv11n** on hand bounding boxes). *Why:* Skeletal hand landmarks and learned convolutional features remain invariant to skin-tone lighting shifts and cluttered backgrounds that break classical colour thresholding.

4. **Post-Processing:** Suppress predictions with confidence below **0.75** and apply a **5-frame rolling majority vote buffer** to prevent the on-screen word from flickering between frames.
5. **Output:** Draw the detected hand bounding box/skeleton and display the recognized word in a high-contrast caption banner at the bottom of the screen.

5. Data

- **Source & Link:** An 8-class subset of the **HaGRID (HAnd Gesture Recognition Image Dataset)** (https://www.kaggle.com/datasets/innominate817/hagrid-sample-30k-384p), combined with custom webcam photos captured by the team.
- **Licence:** CC BY 4.0 (Creative Commons Attribution 4.0 — permitted for academic and research use).
- **Samples per Class: 200 labeled images per class** across 8 word gestures (**1,600 total images**), with sample sets already downloaded locally and 20 labeled samples displayed in our committed contact sheet.
- **How Labels Exist / Are Made:** HaGRID provides pre-verified class labels and normalized bounding boxes ([x, y, w, h]); custom webcam samples are stored in class-specific directories (data/samples/<word_label>/) and verified manually.
- **Conditions Covered:** Indoor artificial and natural lighting, distances from 0.5 m to 2.5 m, both left and right hands, plain walls (easy cases), and cluttered room backgrounds with varied lighting (hard cases).
- **Dataset Split:** Stratified split of **70% Training** (1,120 images), **15% Validation** (240 images), and **15% Test** (240 images), ensuring no subject/frame overlap between train and test splits.

6. Architecture



7. Task Assignments

File	Owner	Responsibilities
src/dataset.py & src/preprocessing.py	Kean Gabriel Salvahan	Data loading, 70/15/15 splitting, Gaussian denoising, colour conversions, and contact sheet script.
src/baseline.py & src/evaluate.py	Prince Genel Umali	YCrCb skin mask thresholding, contour/convexity feature extraction, SVM classifier, and F1/confusion matrix metrics.
src/main_model.py & src/caption_app.py	Justin Angelo Luzande	Hand landmark / deep model training, 5-frame temporal smoothing buffer, and real-time OpenCV captioning HUD.

8. Timeline

#	Checkpoint	Hand in by	Git Tag	Planned Team Deliverables
1	Proposal & Initial plan	Thu 15 Oct, 11:30 PM	cp1-proposal	Proposal PDF, 20-sample labeled contact sheet, 5-image skin/contour feasibility spike.
2	50% progress check	Thu 22 Oct, 11:30 PM	cp2-50	Full 1,600-image 70/15/15 split (dataset.py), preprocessing pipeline, and working **YCrCb Skin Mask + Contour + SVM Baseline**.
3	Core implementation	Thu 29 Oct, 11:30 PM	cp3-core	Trained **MediaPipe Hand Landmarks / CNN Main Method** (main_model.py) and F1/FPS comparison against baseline.
4	80% progress check	Thu 12 Nov, 11:30 PM	cp4-80	Live webcam OpenCV HUD (caption_app.py), 5-frame rolling majority vote filter, and hard-case error analysis.
5 & 6	Final Demo & Defense	Thu 19 Nov, 11:30 PM	v1.0	Code freeze, final evaluation report, live classroom captioning demo, and individual technical defense

9. Risks and Fallbacks

Risk 1 - Background skin-coloured objects (wood furniture, faces, warm walls) corrupt the baseline skin mask**:**

- ***Fallback:*** Enforce a plain background or fixed region-of-interest (ROI) box for the classical baseline, and use hand landmark localization in the main method to ignore background clutter.

Risk 2 - Visual similarity between certain static hand shapes reduces multi-class F1 below 0.85:

- ***Fallback:*** Narrow the vocabulary from 8 word gestures down to the 5 most geometrically distinct static gestures (*Hello, Stop, Yes, No, Thank You*) so the real-time captioning demo remains reliable**.**